# Data Structures and Algorithms - 2025 - 2026



Quiz navigation

1 2 3 4 5
6 7 8 9 10
11 12

Show one page at a time

Finish review

| | |
|---|---|
| **Status** | Finished |
| **Started** | Sunday, 17 May 2026, 3:37 PM |
| **Completed** | Sunday, 17 May 2026, 3:49 PM |
| **Duration** | 12 mins 11 secs |
| **Marks** | 10.00/12.00 |
| **Grade** | **8.33** out of 10.00 (**83.33**%) |

Question **1**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that we have a cuckoo hash table, with m =  7 positions and the following hash functions:

h1(elem) = elem % 7

h2(elem) = (elem / 7) % 7.

After adding some elements, the table looks like that:

| | | | | | | |
|---|---|---|---|---|---|---|
| 14 | | | | 11 | | 62 |
| | | | | | | |

If next we want to insert element 25 to the table, which element(s) will be moved to the second table?

- None of them
- 62
- 11 ✓
- 14
- 25

Your answer is correct.

h1(25) = 4, but position 4 is occupied in the first table. Nevertheless, we put 25 on position 4 in the first table, but we need to move 11 to the second one. According to the second hash function it should go to position 1 and since that is an empty position this is where it will be added.

The correct answer is:
11

Question **2**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that we have a cuckoo hash table, with m =  7 positions and the following hash functions:

h1(elem) = elem % 7

h2(elem) = (elem / 7) % 7.

After adding some elements, the table looks like that:

| | | | | | | |
|---|---|---|---|---|---|---|
| 28 | | | 31 | 25 | 19 | 62 |
| | 11 | 14 | | | | |

If next we want to insert element 47 to the table, which of the already existing element(s) will be moved to a different position?

- None of them
- 62
- 11
- 14 ✓
- 25
- 19 ✓
- 31
- 28 ✓

Your answer is correct.

h1(47) = 5, but position 5 is occupied. So we put 47 on position 5 and move 19 to the second table.

h2(19) = 2, but that position is occupied. So we put 19 on position 2 in the second table and move 14 to the first table.

h1(14) = 0, but that position is occupied. So we put 14 on position 0 and move 28 in the second table.

h2(28) = 4, finally an empty position.

The correct answers are:
14,
19,
28

Question **3**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that we have a cuckoo hash table, with m =  7 positions and the following hash functions:

h1(elem) = elem % 7

h2(elem) = (elem / 7) % 7.

After adding some elements, the table looks like that:

| | | | | | | |
|---|---|---|---|---|---|---|
| 14 | | | 31 | 25 | 47 | 62 |
| | 11 | 19 | | 28 | | |

If next we want to insert element 39 to the table, which of the already existing element(s) will be moved to a different position?

- None of them
- 62
- 11
- 14
- 25 ✓
- 19
- 31
- 28
- 47

Your answer is correct.

h1(39) = 4, but position 4 is occupied. So we put 39 on position 4 and move 25 to the second table.

h2(25) = 3 and that is an empty position, so that is where we put the element 25

The correct answer is:
25

Question **4**
Correct
Mark 1.00 out of 1.00
Flag question

Which collision resolution method could be described as "a hashtable of hashtables"?

- separate chaining
- coalesced chaining
- open addressing
- cuckoo hashing
- linked hashtable
- perfect hashing ✓

Your answer is correct.

The correct answer is:
perfect hashing

Question **5**
Incorrect
Mark 0.00 out of 1.00
Flag question

Assume that we are building a perfect hashing hashtable for storing the following elements: 19, 47, 81, 32, 49, 39, 12, 25, 71.  For the universal hash function we choose the one from the lecture, with p = 101.

For the hash function of the primary hash table, we choose a = 2 and b = 1.

For the secondary hash table from position 7, which values for a and b could NOT be used?

- a = 1, b = 2
- a = 4, b = 5 ✗
- a = 3, b =  11

Your answer is incorrect.

There are a few steps, that you need to compute yourself, to be able to answer this question.

First of all, we add 9 elements, so the size of the primary hash table will be m = 9. This also means that the hash function for the primary hash table is going to be: h(elem) = ((2*elem + 1) % 101) % 9

There are two elements which hash to position 7 in the primary hash table: 39 and 12, so that hash table will have 4 positions (square of number of elements) and the corresponding hash function will be: h(elem) = ((a * elem + b) % 101) % 4

Now we only have to check for what values of a and b, there is no collision in the table:

a = 1, b = 2 => h(39) = 1, h(12) = 2 => OK

a = 3, b = 11 => h(39) = 3, h(12) = 3 => NOT OK

a = 4, b = 5 => h(39) =0, h(12) = 1 => OK

The correct answer is:
a = 3, b =  11

Question **6**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that we are building a perfect hashing hashtable for storing the following elements: 19, 47, 81, 32, 49, 39, 12, 25, 71.  For the universal hash function we choose the one from the lecture, with p = 101.

For the hash function of the primary hash table, we choose a = 2 and b = 1.

How many positions are we going to have, in total in all the primary and secondary hash tables?

- 81
- 18
- 22 ✓
- 9

Your answer is correct.

There are a few steps, that you need to compute yourself, to be able to answer this question.

First of all, we add 9 elements, so the size of the primary hash table will be m = 9. This also means that the hash function for the primary hash table is going to be: h(elem) = ((2*elem + 1) % 101) % 9

h(19) = 3

h(47) = 5

h(81) = 5

h(32) = 2

h(49) = 0

h(39) = 7

h(12) = 7

h(25) = 6

h(71) = 6

So we have 9 positions for the primary hash table.

On position 0 we have  1 element => secondary table of size 1

On position 1 we have 0 elements => no secondary table

On position 2 we have 1 element => secondary table of size 1

On position 3 we have 1 element => secondary table of size 1

On position 4 we have 0 elements => no secondary table

On position 5 we have 1 element => secondary table of size 1

On position 6 we have 2 elements => secondary table of size 4

On position 7 we have 2 elements => secondary table of size 4

On position  we have 1 elements => secondary table of size 1

Total: 9 + 1+ 1+ 1+ 1+ 4 + 4 + 1 = 22 positions

The correct answer is:
22

Question **7**
Correct
Mark 1.00 out of 1.00
Flag question

To what collision resolution method is closest the linked hash table?

- separate chaining ✓
- coalesced chaining
- open addressing

Your answer is correct.

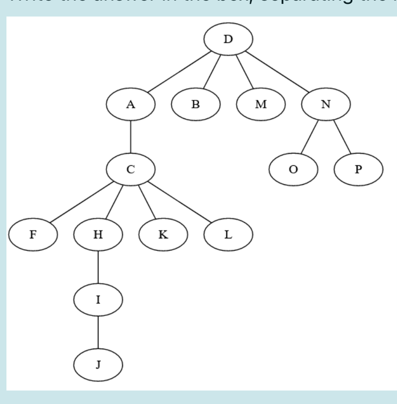
Linked hash tables are essentially separate chaining with some extra links.

The correct answer is:
separate chaining

Question **8**
Correct
Mark 1.00 out of 1.00
Flag question

In what order will be visited the nodes of the following tree if we use Breadth First (Level order) Traversal? Consider nodes from left to right.

Write the answer in the box, separating the nodes by a space.




Answer: D A B M N C O P F H K L I J ✓

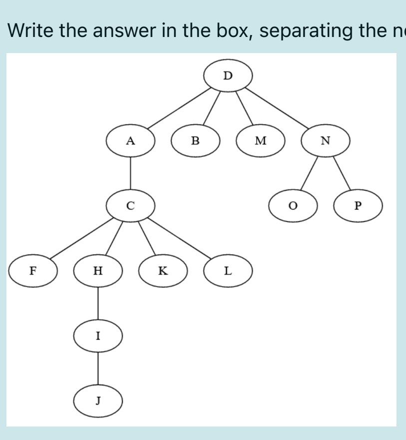
Breadth first traversal means to visit all nodes on a level and then continue with the next level. So the order will be: D A B M N C O P F H K L I J

The correct answer is: D A B M N C O P F H K L I J

Question **9**
Correct
Mark 1.00 out of 1.00
Flag question

In what order will be visited the nodes of the following tree if we use Depth First Traversal? Consider nodes from left to right.

Write the answer in the box, separating the nodes by a space.




Answer: D A C F H I J K L B M N O P ✓

Depth first traversal means to go in depth (down) as much as possible and when it is no longer possible, return and go on the next child. So the order will be: D A C F H I J K L B M N O P

The correct answer is: D A C F H I J K L B M N O P

Question **10**
Incorrect
Mark 0.00 out of 1.00
Flag question

What is the maximum number of nodes in a 3-ary tree (a tree in which every node can have at most 3 children), where the order of the nodes in the depth first traversal is the same as the order of the nodes in the level-order traversal?

- 1
- 2 ✗
- 3
- 4
- 5
- More than 5, but limited
- It is unlimited (there is no maximum value)

Your answer is incorrect.

If you have one single node, the two traversals are obviously the same.

For two nodes (root + 1 child) the two traversals are the same as well.

If we add 3 children to the root, the two traversals will still be the same.

If we add 3 children to the root and let's say, the first will have another child, the two traversals will no longer be the same.  But if you put that extra node to be a child of the 3rd child of the root, the two traversals will be the same again.

So we have seen that it is possible for 5 nodes.

Actually, it is possible for even more, if we add just one child to every node (including the root): this creates a chain (does not really look like a tree), but this chain will have the same depth first and level order traversal. And we can add an unlimited number of nodes in this tree.

The correct answer is: It is unlimited (there is no maximum value)

Question **11**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that you have a 3-ary tree (a tree in which every node has at most 3 children) with 20 nodes.

What is the minimum height of the tree?

Answer: 3 ✓

The minimum height can be achieved if every node has as many children as possible. We have the root + 3 children + 3x3 children = 13 nodes for a height of 2. The remaining 7 nodes can "fit" anywhere on the next level (next level could contain 21 nodes). So the height is 3.

The correct answer is: 3

Question **12**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that you have a 3-ary tree (a tree in which every node has at most 3 children) with 20 nodes.

What is the maximum height of the tree?

Answer: 19 ✓

Maximum height can be achieved when every node has one single child.

The correct answer is: 19

Finish review

◄ Lecture 10 Quiz

Jump to...